

```

-----+-----+-----+-----+-----+-----+-----
----
forum | perm_test | table |           |           |
(1 row)

```

Since there is no ACL associated with the table, how can PostgreSQL know what roles are permitted to do what on the object? The answer lies in the **default privileges**: PostgreSQL applies a set of default privileges to the object and checks against its default list.

Most notably, if the role is the owner of the object, it has all the available privileges for such an object. If the role is not the owner, the PUBLIC permissions are inspected, that is, all permissions assigned to the special PUBLIC role for that kind of object are used.

The list of PUBLIC associated privileges is quite short, for security reasons, and can be summarized as:

- Execute permission (X) on routines
- Connect to and create temporary objects on databases (cT)
- Use of languages, types, and domain (U)

As you can see, by default, the PUBLIC set of privileges does not allow a role to do anything really dangerous, and therefore the only way to authorize a role to perform actions against objects is to GRANT and REVOKE permissions carefully.

The first time GRANT is performed against an object, PostgreSQL also introduces the default ACL for the owner of that object. In the case of the preceding table, foo, the owner will have an ACL such as luca=arwdDxt/luca (assuming the luca role is the owner). So, suppose we give permissions to manipulate data to enrico:

```

forumdb=> \dp perm_test

                                Access privileges
Schema | Name      | Type | Access privileges | Column privileges |
Policies
-----+-----+-----+-----+-----+-----+-----
----
forum | perm_test | table |           |           |
(1 row)

forumdb=> GRANT SELECT, INSERT,
          UPDATE, DELETE

```

```

        ON perm_test TO enrico;

GRANT
forumdb=> \dp perm_test

                Access privileges
Schema |   Name   | Type | Access privileges | Column privileges |
Policies
-----+-----+-----+-----+-----+-----+-----
forum  | perm_test | table | forum=arwdDxt/forum+ |                  |
        |           |       | enrico=arwd/forum   |                  |
(1 row)

```

As you can see, after GRANT, the ACL is made by two entries, the one we just granted to the user `enrico`, and the one that was implicitly applied to the table owner `luca`.

It is also important to note that ACLs store what a role can do, not what it cannot do. Everything not listed in the ACLs is rejected. To better understand this, consider revoking permission for the `enrico` role:

```

forumdb=> REVOKE TRUNCATE ON perm_test FROM enrico;
REVOKE

forumdb=> \dp perm_test

                Access privileges
Schema |   Name   | Type | Access privileges | Column privileges |
Policies
-----+-----+-----+-----+-----+-----
forum  | perm_test | table | forum=arwdDxt/forum+ |                  |
        |           |       | enrico=arwd/forum   |                  |
(1 row)

```

As you can see, this revocation did not change the ACL line for the `enrico` role. The role did not have this permission; therefore, revoking it had no effect on the ACLs.

Similarly, revoking permissions for `PUBLIC` does not affect already existing ACLs. If we remove the `INSERT` permission from every user, `enrico` will still retain his own permission because ACLs are stored additively:

```

forumdb=> REVOKE INSERT ON perm_test FROM PUBLIC;

```

```
REVOKE
```

```
forumdb=> \dp perm_test
```

```

                                Access privileges
Schema |   Name   | Type | Access privileges | Column privileges |
Policies
-----+-----+-----+-----+-----+-----+-----
forum  | perm_test | table | forum=arwdDxt/forum+ |                  |
      |          |      | enrico=arwd/forum  |                  |
(1 row)
```

To summarize, ACLs are always empty for a freshly created object. In this situation, the object owner has all available permissions and other roles have the default permissions associated with PUBLIC. The first GRANT or REVOKE statement executed against that object will create also the explicit owner ACL and, in the case of GRANT, will add another one accordingly.

ACLs are stored as granted privileges. What is not explicitly set in an ACL is implicitly rejected as it has been revoked.

Knowing the default ACLs

It is now clear that the owner of an object has all the possible permissions related to such an object. But what about other roles? It is possible to inspect the default ACL provided once an object is instantiated via the special function `acldefault`.

The function accepts two arguments – a type of object (for example, a relation/table, a function, etc.) and the OID value of the role that is supposed to create the object. The function will return the ACLs that will be in place after the creation of the object.

For example, in order to see the permissions provided when your role creates a new table (type `r`, for relation), you can perform the following query:

```

forumdb=> SELECT acldefault( 'r', r.oid )
          FROM pg_roles r
          WHERE r.rolname = CURRENT_ROLE;
acldefault
-----
{ forum=arwdDxt/forum}
```

```
(1 row)
```

Nothing new here, but what about the creation of a function (type f)? It is now easy to see the following:

```
forumdb=> SELECT acldefault( 'f', r.oid )
           FROM pg_roles r
           WHERE r.rolname = CURRENT_ROLE;
acldefault
-----
{=X/forum,forum=X/forum}
(1 row)
```

This time, two ACLs are produced: the first grants all users the executable permission, while the latter specifies that the owner is the forum role with executable permissions, too.

You can inspect all the default ACLs for a specific user by means of its OID and the type of object, where the main types are *r* for tables, *c* for columns, *l* for languages, and *f* for routines and procedures. Other types are available. Please refer to the official documentation. It is now time to see how to manipulate ACLs and permissions in a practical way. In the next section, you will learn how to deal with permission management.

Granting and revoking permissions

As you saw in Chapter 3, *Managing Users and Connections*, a role is associated with a collection of permissions, which are provided by means of a GRANT statement and removed by means of a REVOKE statement. Permissions are stored internally as ACLs, as you saw in the previous section.

This section revisits the GRANT and REVOKE statements to better help you understand how to use them, with respect to different database objects.

The GRANT statement has the following synopsis:

```
GRANT <permission, permission, ...> ON <database-object> TO <role>;
```

Here, you list all the permissions you want to associate with the target role for the specified database object. It is also possible to extend the GRANT statement with the WITH GRANT OPTION clause, which will cause the target role to be able to grant the same permissions it has received to another role.

The REVOKE statement has a similar synopsis:

```
REVOKE <permission, permission, ...> ON <database-object> FROM <role>;
```

There is a special role, named PUBLIC, that can be used when dealing with permission management. It is not a concrete role, but rather a marker to indicate “all available roles.” In other words, if you grant a permission to PUBLIC, you are implicitly granting this permission to all available roles.

But what does “all available roles” mean? It means all existing and future roles. The PUBLIC role represents any role that will ever be present in the system, at the time the permission is managed and in the future.

According to the above, in order to prevent any user from accessing your objects, you should always remove all the permissions from the special PUBLIC role, and then selectively provide the permissions you need for specific roles.

In the following sections, we will detail different permissions for assigning and removing groupings and classify them depending on the database object. As a general rule of thumb, the list of permissions depends on the action you can run against the database object.

In many cases, the special keyword ALL is a substitute for every permission related to the database object.

Permissions related to tables

We already saw the main permissions related to a database table. They refer to the main statements that can run against a table object, such as SELECT, INSERT, UPDATE, DELETE, and TRUNCATE. Moreover, it is possible to use the special keywords TRIGGER and REFERENCES to create triggers and foreign keys within a table.

Of course, the special keyword ALL does include all the preceding permissions.

As an example, in order to provide the forum_stats role with the permissions to read, update, and insert data into the categories table, without granting permissions to execute the other actions, you can do the following once connected as the forum user:

```
forumdb=> REVOKE ALL
           ON forum.categories FROM forum_stats;

REVOKE

forumdb=> GRANT SELECT, INSERT, UPDATE
```

```

        ON forum.categories TO forum_stats;

GRANT

forumdb=> \dp categories

Access privileges
Schema | Name      | Type | Access privileges | Column privileges | Policies
-----+-----+-----+-----+-----+-----
forum  | categories | table | forum=arwdDxt/forum +|                  |
        |            |      | forum_stats=arw/forum |                  |
(1 row)

```

The first REVOKE statement is not mandatory, but it is a good practice. Since we want to ensure that the role has precisely the permissions we are going to grant and not anything more, removing all the permissions from the role ensures that any previous GRANT statements will not persist.

As you can see, the ACL for the `forum_stats` user reflects the permissions we granted.

Column-based permissions

Since certain statements related to table objects can address columns directly, for example, `SELECT` and `UPDATE`, it is also possible to grant or revoke column permissions. The synopsis is the same, but you can list the columns that the permission refers to.

Column privileges can be applied only to `SELECT`, `UPDATE`, `INSERT`, and `REFERENCES` permissions because those are the ones that can refer to columns explicitly; the special keyword `ALL` encapsulates the entire list of permissions.

As an example, consider a scenario where the `forum_stats` user can interact with table users only via the `gecos` and `username` columns, being able to read both of them but update just the first one. The permissions could be assigned by the user `forum` as follows:

```

forumdb=> REVOKE ALL ON forum.users
        FROM forum_stats;

REVOKE

forumdb=> GRANT SELECT (username, gecost),
        UPDATE (gecost)
        ON forum.users TO forum_stats;

GRANT

```

As already stressed, it is a good practice to include the first REVOKE statement to ensure that the permissions for the role are reset before we assign the ones we want. Then, we grant the SELECT and UPDATE permissions, specifying the columns every statement will be able to interact with.

The side effect of the preceding GRANT statement is that the forum_stats role is no longer able to issue SELECT or UPDATE with a column list wider than the one specified in GRANT:

```

forumdb=> SELECT current_role;
current_role
-----
luca
(1 row)

-- denied, not all the columns can be read!
forumdb=> SELECT * FROM forum.users;
ERROR:  permission denied for table users

-- allowed
forumdb=> SELECT gecos, username FROM forum.users;
      gecos      | username
-----+-----
 LUCA FERRARI   | fluca1978
 ENRICO PIROZZI | sscotty71
(2 rows)

-- denied, the 'username' column cannot be updated!
forumdb=> UPDATE users SET username = upper( username );
ERROR:  permission denied for table users

-- allowed
forumdb=> UPDATE forum.users SET gecos = lower( gecos );
UPDATE 2

```

Let's now inspect the permissions for the users table:

```

forumdb=> \dp forum.users
Access privileges

```


If we inspect the permissions after such a statement, we can see that the ACL has been inserted as an access privilege:

```
forumdb=> \dp users
```

Schema	Name	Type	Access privileges	Column privileges
forum	users	table	forum=arwdDxt/forum+ forum_stats=r/forum	username: forum_stats=r/forum gecos: forum_stats=rw/forum

(1 row)

Which permission will be considered in the case of a SELECT statement?

It is easy to test and see that PostgreSQL considers the last granted permission more open than the one granted to the column. Therefore, the role has been granted the ability to select every column on the table:

```
forumdb=> SELECT * FROM users;
```

pk	username	gecos	email
1	fluca1978	luca ferrari	fluca1978@gmail.com
2	sscotty71	enrico pirozzi	sscotty71@gmail.com

(2 rows)

Fixing the problem may not be as simple as you think. Revoking read permission on the columns you don't want the role to have access to may not do what you expect, even if done by the table owner (the forum user):

```
forumdb=> REVOKE SELECT (pk, email)
          ON users FROM forum_stats;
REVOKE
```

If you remember, REVOKE does not store an ACL but modifies existing ones. In this particular case, since there is nothing related to the preceding pk and email columns, the REVOKE statement does not change anything:

```
forumdb=> \dp users
```

```

forumdb=> \dp users

```

		Access privileges		
Schema	Name	Type	Access privileges	Column privileges
forum	users	table	forum=arwdDxt/forum+ forum_stats=r/forum	username: + forum_stats=r/forum + gecos: + forum_stats=rw/forum

```

(1 row)

```

The rule of thumb is that every specific GRANT statement is canceled by the counterpart, REVOKE. In this example, since the last GRANT statement was issued without a specific list of columns, we need to issue a REVOKE statement from the forum user without the list of columns:

```

forumdb=> REVOKE SELECT
          ON users FROM forum_stats;
REVOKE

```

However, this also removes the column-based grant permissions, so after REVOKE, the forum_stats role will no longer be able to perform SELECT against the username and gecoss columns. In order to re-enable the role, you must re-issue the GRANT statement for the targeted columns.

The preceding example showed you that the application of permissions at a fine-grain level requires attention and care because an overly wide GRANT or REVOKE statement can produce results you would not expect at a glance.

Permissions related to sequences

A sequence is a table-like object that produces a transaction-safe stream of new values, usually used for autogenerated (synthetic) keys.

There are three main permissions associated with a sequence: USAGE allows the querying of new values from the sequence; the SELECT privilege allows querying of the last or current value from the sequence (but not getting a new one); and lastly, the UPDATE privilege is another PostgreSQL-specific extension that allows the value of the sequence to be set and/or reset.

Since the USAGE privilege is the only one recognized by the SQL standard, if you grant it to a role, that role will automatically be able to perform actions that require SELECT and UPDATE privileges.